DISTRIBUTED SYSTEMS · FA-1 + FA-2 · CONCEPT REFERENCE

Campus Mesh Explained

All four units, matched to the line of code that implements them and the screen that proves it. Units 1 and 2 are built and running; Units 3 and 4 are designed against the same system for FA-2. Every block is marked ***built*** *or* ***planned****, so nothing here claims more than it should.*

CONTENTS

1 The system

2 Is it really distributed?

3 Unit 1 — Introduction

4 Unit 2 — Communication

5 Traces & time

6 How FA-1 was built

7 Unit 3 — Synchronization

8 Unit 4 — Emerging paradigms

9 The FA-2 build plan

10 Demo script

11 Viva questions

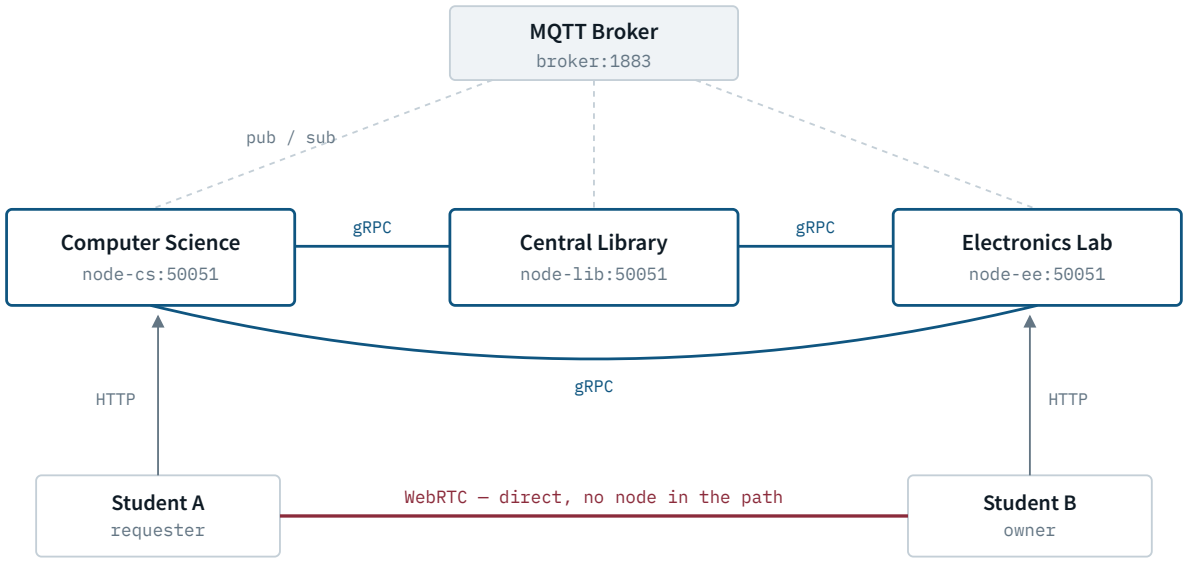
12 Limitations

1 The system

What it does, in one paragraph, before any theory.

A student needs an HDMI cable for two hours. Instead of asking a WhatsApp group, they type it into their own department's node. That node checks its own inventory, asks every other node in parallel, and returns one merged list. The student never says *where* to look. When they request an item, the request is routed to the node that owns it — the only node allowed to decide — and once granted, the two students get a direct browser-to-browser channel, text and live video, to arrange the physical handover.

Three departments run as three autonomous nodes. Each owns its inventory outright. There is no shared database, no central server, and no coordinator.



Solid blue = synchronous RPC mesh. Dashed = asynchronous publish/subscribe. Red = peer-to-peer data and media. Three communication styles, three visually distinct paths.

2 Is it really distributed?

The first question an examiner asks about three programs on one laptop. Four properties answer it.

PROPERTY	HOW CAMPUS MESH SATISFIES IT
No shared state	Each node loads its own `data/seed.<node>.json` into its own process memory. No shared database, no shared file, no shared memory.
Communication only by message	A node can discover another node's inventory *only* by sending it a gRPC call or hearing an MQTT announcement. There is no back door.
Independent failure	Any node can die without stopping the others. Proven: `docker stop node-ee` and the mesh keeps answering.
No global clock	Ordering comes from Lamport logical clocks, not wall time, because three machines' clocks disagree.

SAY THIS

"Containers give each node its own network identity and filesystem. The topology is unchanged if they move to three laptops — only the `PEERS` addresses change, not one line of logic."

3 Unit 1 — Introduction

BUILT & RUNNING · FA-1

Introduction, architectures, design issues, middleware, multimedia, virtualization, models of computation.

DEFINITION & GOALS

What a distributed system is for

A distributed system is a collection of independent computers that appears to its users as a single coherent system. The standard goals are **resource sharing**, **transparency**, **openness** and **scalability**.

Resource sharing is not an abstraction here — it is the literal subject. The resources are physical objects owned by different departments that will not surrender them to a central authority.

> IN CAMPUS MESH
>
> The system exists because three departments each hold equipment and none will hand its inventory to a central server. That constraint is what forces a distributed design rather than a database with three logins.

TYPES OF DISTRIBUTED SYSTEMS

Which type this is

The three classic categories are **distributed computing systems** (clusters, grids — built for throughput), **distributed information systems** (transaction and enterprise integration — built for consistency), and **distributed pervasive systems** (mobile and embedded — built for volatility).

Campus Mesh is a **distributed information system**, with one pervasive characteristic bolted on: nodes join and leave without warning, so the system must tolerate churn rather than assume a fixed membership.

> IN CAMPUS MESH
>
> Reservations are the transactional part — exactly one student may hold an item. The churn shows up in the heartbeat: a node's peers are discovered as `UP` or `DOWN` at runtime, never assumed.
>
> src/rpc/peers.js · startHeartbeat()

ARCHITECTURES

A deliberately hybrid architecture

Architectural style describes how components are organised and how they interact. Campus Mesh uses four styles at once, each where it fits, and being able to name each edge of the diagram is what a system-design mark is made of.

EDGE	STYLE	WHY THERE
browser ↔ its own node	Client–server	The browser is a thin consumer; the node holds the data and the logic.
node ↔ node	Structured P2P overlay	Every node both serves and consumes the same service. Genuinely symmetric, not client–server in disguise.
node → all nodes	Publish–subscribe / event-based	An announcement has no addressee, so request–reply is the wrong shape.
browser ↔ browser	Pure peer-to-peer	Two students, not two nodes. Routing through a server adds latency and a failure point for no benefit.
inside one node	Layered	UI → HTTP → middleware → store, each layer ignorant of the one above.

SAY THIS

"It is symmetric, not client–server with extra steps: `node-ee` serves `Search` to `node-lib` in one breath and calls `Search` on `node-lib` in the next, using the same stub and the same `.proto`."

DESIGN ISSUES

Transparency, openness, scalability

Transparency means hiding the fact that resources are physically distributed. There are several kinds; three are demonstrable on screen rather than merely claimed.

Access transparency

Local and remote results render identically. Only the terminal log reveals that one crossed the network. The reply is one flat list of uniformly shaped records.

Location transparency

The browser sends `{ query: "hdmi" }` and nothing else. It cannot name a node, and it does not learn which node held the item until it chooses to request one.

Failure transparency — partially, and deliberately so

A dead node does not break a search. But the result line says *"2 of 3 nodes responded"* rather than pretending everything is fine. Full failure transparency would be a lie here: the student is entitled to know the answer may be incomplete.

> IN CAMPUS MESH
>
> **Openness** is the `.proto` file. It is a published interface definition; a fourth node joins the network by implementing it, and no existing node changes. That is the textbook definition of an open system.
>
> **Scalability** is the parallel fan-out. Search latency is bounded by the *slowest* peer, not the *sum* of all peers, because every peer is asked at the same time.
>
> `proto/resource.proto · src/rpc/peers.js · src/http/api.js`

BE HONEST ABOUT THIS

Replication transparency and concurrency transparency are *not* implemented. There are no replicas, and two simultaneous requests are serialised by the single owning node. Those are Unit 3 and 4 topics — claim them and you will be caught.

MIDDLEWARE

What the middleware layer hides

Middleware sits between the operating system and the application, offering a uniform programming model and hiding the heterogeneity and the network beneath it.

The application code in Campus Mesh calls `distributedSearch("hdmi")`. It never touches a socket, never serialises a byte, never retries, never resolves a hostname, and never handles a partial failure directly. Everything in that list is done by the middleware layer.

CONCERN	HIDDEN BY
Marshalling and unmarshalling	gRPC + Protocol Buffers
Connection management, HTTP/2 multiplexing	gRPC channel
Name resolution at call time	gRPC + container DNS
Timeouts and cancellation	gRPC deadlines
Topic matching and fan-out delivery	MQTT broker
NAT traversal, encryption, congestion control	WebRTC / ICE / DTLS

> IN CAMPUS MESH
>
> Point at two files when asked where the middleware is: `src/rpc/peers.js` and `src/mq/bus.js`. Everything above them speaks in resources and queries; everything below them speaks in bytes.

DISTRIBUTED MULTIMEDIA SYSTEMS

Continuous media and quality of service

A multimedia distributed system carries **continuous media** — data whose meaning depends on *when* it arrives, not only on arriving correctly. This creates requirements no discrete system has: bounded end-to-end delay, bounded jitter, and a policy for what to do when the network cannot keep up.

> IN CAMPUS MESH
>
> **"Verify before you walk."** Before crossing campus for a cable, the requester watches a live feed of it from the owner's camera. The QoS panel reports frames per second, jitter, packets lost, frames dropped and bitrate, read live from `RTCPeerConnection.getStats()`.
>
> Throttle the network in DevTools and every number degrades while the picture keeps moving — adaptation, not failure.
>
> `public/rtc.js · startStats()`

SAY THIS WHILE THE NUMBERS MOVE

"An RPC reply that arrives late is still correct. A video frame that arrives late is worthless — so the stream discards it rather than delaying the frames behind it. That trade-off is the whole difference between discrete and continuous communication."

SELF STUDY VIRTUALIZATION

Why containers, not just three terminals

Virtualization provides an isolated execution environment. Containers are OS-level virtualization: isolated process tree, filesystem and network namespace, sharing one kernel — far lighter than a virtual machine, which emulates hardware and runs a whole guest OS.

That lightness is exactly why three "departments" fit on one laptop at all.

> IN CAMPUS MESH
>
> One image becomes three nodes. Every node listens on gRPC **50051 inside its own container** — they are distinguished by *hostname*, not by port, and Compose supplies the DNS. That makes `node-ee:50051` a real address resolved fresh on each call.
>
> It also upgrades the failure demo from "close a terminal" to `docker stop node-ee`, which is a far more convincing kill.
>
> Dockerfile · docker-compose.yml

SELF STUDY · MODEL OF DISTRIBUTED COMPUTATIONS

Events, causality and logical time

A distributed computation is modelled as a set of events on each process — *send*, *receive* and *internal* — partially ordered by Lamport's **happened-before** relation:

- If a and b are on the same process and a comes first, then $a \rightarrow b$.
- If a is a send and b the matching receive, then $a \rightarrow b$.
- The relation is transitive.

Events unrelated by $\rightarrow$ are **concurrent**. A Lamport clock assigns numbers respecting this: if $a \rightarrow b$ then $L(a) < L(b)$. The converse does not hold — which is exactly the limitation that motivates vector clocks in FA-2.

src/clock.js

```
class LamportClock {
  constructor() { this.t = 0; }
  tick()          { return ++this.t; }              // local or send event
  receive(remote) {                                   // receive event
    this.t = Math.max(this.t, Number(remote || 0)) + 1;
    return this.t;
  }
}
```

> IN CAMPUS MESH
>
> Every gRPC message and every MQTT message carries a `lamport` field, and every log line prints the clock in a fixed column. Section 5 turns a real trace into a space–time diagram.

4 Unit 2 — Communication

BUILT & RUNNING · FA-1

Communication: RPC, message-oriented, stream-oriented, P2P, WebRTC, naming, fault tolerance.

FUNDAMENTALS OF COMMUNICATION

Two axes that classify every channel

Communication is layered — the application never speaks to the wire directly — and any channel can be placed on two independent axes.

- **Synchronous vs asynchronous:** does the sender block until the receiver responds?
- **Transient vs persistent:** is the message discarded if the receiver is not available, or stored until it can be delivered?

CHANNEL	SYNC?	PERSISTENCE	COUPLING
gRPC	Synchronous	Transient	Caller must know the callee
MQTT	Asynchronous	Transient at QoS 0	Publisher knows nobody
WebRTC data	Asynchronous	Transient	Direct peer link
WebRTC media	Continuous	Transient by design	Direct peer link

PRECISION MATTERS HERE

Campus Mesh publishes at MQTT **QoS 0** with no retained messages, so announcements are *transient*: a node that is offline when an item is returned never hears about it. Calling this "persistent messaging" is wrong, and it is an easy question to be caught by. Persistence would need QoS 1 plus a durable session — a clean FA-2 addition.

REMOTE PROCEDURE CALL

RPC: making a network call look local

RPC lets a program call a procedure in another address space using ordinary call syntax. The machinery that makes that possible:

1. The **client stub** accepts a normal call and **marshals** the arguments into a message.
2. The message travels to the **server skeleton**, which **unmarshals** and invokes the real procedure.
3. The result is marshalled back, and the stub returns it as a normal return value.

Both sides are generated from an **interface definition** — the contract. Campus Mesh uses gRPC with Protocol Buffers, so the IDL is a real artefact you can print.

proto/resource.proto — the published interface

```
service ResourceNode {
  rpc Search  (SearchRequest)  returns (SearchReply);
  rpc Reserve (ReserveRequest) returns (ReserveReply);
  rpc Ping    (PingRequest)    returns (PingReply);
}
```

Why RPC cannot be perfectly transparent

A local call cannot fail because the callee is unreachable; a remote one can. So RPC must expose failure semantics that local calls do not have:

- **Maybe** — no guarantee at all.
- **At-least-once** — retried until acknowledged; unsafe unless the operation is idempotent.
- **At-most-once** — never executed twice, but may not execute at all.

> IN CAMPUS MESH
>
> `Search` is idempotent and safe to retry. `Reserve` is *not* — a blind retry could double-book — so it is never retried, giving **at-most-once** semantics. A failed `Reserve` surfaces as a clean 503 and the student decides what to do. Guessing on the student's behalf is what would cause the bug.
>
> src/rpc/server.js · src/rpc/peers.js · src/http/api.js

The parallel fan-out

This is the core of the whole project. Every peer is asked at the same time, each with its own deadline, and partial results are accepted.

src/rpc/peers.js

```
const settled = await Promise.allSettled(
  peers.map((p) => askPeer(p, query))    // all at once
);

const ok = settled.filter((r) => r.status === 'fulfilled');
// latency is bounded by the SLOWEST peer, not the SUM  -> scalability
// a dead peer costs one deadline and never blocks       -> fault tolerance
```

MESSAGE-ORIENTED COMMUNICATION

Publish/subscribe and three kinds of decoupling

Message-oriented middleware replaces "call this node and wait" with "put this message on a topic". The publisher does not block, does not name a recipient, and never learns who received it. That buys decoupling along three dimensions:

- **In space** — publisher and subscriber need not know each other's identity or address.
- **In time** — they need not be active in the same instant.
- **In synchronisation** — neither blocks on the other.

> IN CAMPUS MESH
>
> Topics:
>
> ```
> topic scheme
>
> campus/resource/available/<category>   # an item was returned
> campus/resource/reserved/<category>    # an item was taken
> campus/signal/<room>                   # WebRTC signalling relay
> ```
>
> The broker in `broker/` is deliberately dumb. It knows nothing about resources, students or nodes — it forwards by topic and nothing else. That ignorance *is* the decoupling.
>
> src/mq/bus.js · broker/index.js

The case request–reply cannot serve

A student searches for a VR headset and the whole mesh has none. There is no node to send a request to. Their need is instead **parked** in the wantlist on their own node, and an announcement from *any* node later wakes it up. Over RPC, that node would have to poll every peer forever.

TWO DESIGN DECISIONS WORTH DEFENDING

The store emits events; it does not publish. `store.js` emits `reserved` / `released`, and `src/index.js` is the only place those events become MQTT messages. So both the local HTTP path and an inbound RPC from another node announce a change without either knowing the other exists.

Own announcements are not filtered out. A different student on the same node may want exactly that item, and a publisher has no idea who is listening. Suppressing the echo would be treating pub/sub as if it were addressed messaging.

STREAM-ORIENTED COMMUNICATION

When timing is part of correctness

Discrete communication cares only that a message arrives intact. Stream-oriented communication carries data where **temporal relationships between items are part of the meaning**. Playing frames at the wrong intervals corrupts the content just as surely as losing bytes.

REQUIREMENT	MEANING	WHERE YOU SEE IT
Throughput	Bits per second the stream needs	`bitrate` in the QoS panel
Delay	End-to-end latency	Felt directly in the video
Jitter	*Variation* in delay — often worse than delay itself	`jitter`, in milliseconds
Loss	Packets or frames that never arrive	`lost` and `dropped`

Jitter deserves emphasis. A steady 200 ms delay is watchable; a delay swinging between 20 ms and 400 ms is not. This is why receivers buffer — trading a little more delay for a lot less jitter.

> IN CAMPUS MESH
>
> The owner's camera streams to the requester so they can confirm the item exists and works before walking. Crucially, a lost video frame is **never retransmitted** — by the time a replacement arrived its moment would have passed. Compare with `Search`, where a late reply is still perfectly good.

PEER-TO-PEER MESSAGING

Two peer relationships, at two levels

In a peer-to-peer system every participant is both client and server; there is no permanent asymmetry of role. Overlays are **structured** (membership and routing follow a defined topology) or **unstructured** (random neighbours, flooding).

> IN CAMPUS MESH
>
> **Node level:** a structured overlay — a full mesh of three, where each node both serves and consumes `ResourceNode`. A search is flooded exactly one hop with `ttl: 1`, and a node never forwards a search that arrived by RPC. With three nodes in a full mesh, forwarding would loop within milliseconds.
>
> **Student level:** true peer-to-peer over WebRTC — two browsers exchanging data with no server in the path at all.

WEBRTC

Signalling is not the data path

The single most important idea: WebRTC standardises the *media and data path* but deliberately specifies **no signalling protocol**. Getting the two peers introduced is the application's problem, and any channel will do.

PIECE	JOB
SDP	Session description — codecs, tracks, parameters. Exchanged as offer and answer.
ICE	Tries every candidate address pair to find one that actually works.
STUN	Tells a peer its own public address as seen from outside its NAT.
TURN	Relays traffic when no direct path exists. *Not configured here.*
DTLS / SRTP	Encryption. Mandatory in WebRTC, not optional.

The interesting problem in this project

The two students are attached to *different nodes*. So the offer, answer and ICE candidates must cross the mesh before the peer connection can exist. Rather than adding a signalling server, the existing message bus carries them on `campus/signal/<room>`. Each browser POSTs to its own node; every node receives the message and pushes it to whichever of its browsers are in that room. **The nodes never parse the payload** — they are an opaque relay.

Room ids are derived from the reservation itself (`room-<rid>-v<version>`), so both sides compute the same room without negotiating one.

Role asymmetry, and why it is not arbitrary

The **owner is always the offerer**. The owner is the side that later switches on a camera, and adding a track fires `negotiationneeded` on whoever holds it. Keeping every offer on one side means renegotiation can never collide with an offer travelling the other way — there is no *glare* to resolve.

THE PROOF, AND THE DEMO MOMENT

Verified by driving two browsers against two different nodes: both reached `connectionState: connected` on one candidate pair with mirrored byte counters — 2331 bytes sent on one side, 2331 received on the other.

Then **both node processes and the broker were killed**. `fetch('/api/node')` fails on both browsers, and a message sent after that still arrives, with byte counters still climbing. Nothing but a genuine browser-to-browser link can behave that way.

SELF STUDY · NAMES, IDENTIFIERS, ADDRESSES

Three different things, routinely confused

CONCEPT	PROPERTY	IN CAMPUS MESH
Name	Human-facing, may change, need not be unique	`"Electronics Lab"`
Identifier	Refers to exactly one entity, never reused, never changes	`node-ee`
Address	Where an entity currently is; can move	`node-ee:50051`

Resources use the same scheme: `node-ee:cable:0001` — owner, category, instance.

> WHY THIS EARNS MARKS RATHER THAN BEING TRIVIA
>
> Because the owner is *encoded in the identifier*, reservation routing needs no directory service at all. The node splits the `rid`, and if the owner is not itself, calls that node by RPC. Identity determines routing.
>
> A node holds nothing about itself in code — name, identifier and peer addresses all arrive from the environment, which is what lets one image become three nodes. The lab could move buildings and get a new address while its identifier stays put.
>
> src/config.js · src/http/api.js

A DETAIL WORTH TELLING

During development all three nodes ran on one host, so every peer address began `localhost` and all three derived the *same* identifier — which broke reservation routing. The fix was an explicit `node-ee@localhost:50053` form separating identity from address. A bug that is itself the lesson.

SELF STUDY FAULT TOLERANCE

Detecting failure, and being honest about it

A **fault** is a defect, an **error** is the wrong state it causes, and a **failure** is the system failing to deliver. Failure types, in rising difficulty:

- **Crash** — the node halts. Handled here.
- **Omission** — a message is lost. Handled by deadlines.
- **Timing** — a response arrives too late. Handled by deadlines.
- **Response** — a wrong answer. Not handled.
- **Arbitrary / Byzantine** — arbitrary or malicious behaviour. Not handled, and out of scope.

> THREE LAYERED MECHANISMS
>
> - **Deadlines.** Every RPC carries a 2-second budget, so a dead peer costs one deadline instead of hanging the search.
> - **Heartbeat.** `Ping` every 3 s; three consecutive misses marks a peer `DOWN`, one success restores it. Liveness is tested on the *same channel* a search uses — a peer that cannot answer `Ping` cannot answer `Search` either. And three misses, not one, because a single missed beat is a GC pause, not a failure.
> - **Honest partial results.** The reply carries `responded` and `total`, and the UI says so whenever they differ.
>
> `src/rpc/peers.js · startHeartbeat()`

Note the distinction between **masking** a failure and **reporting** it. Masking is right when a replica can serve the same answer. Here no replica exists, so masking would mean silently returning an incomplete result — and a student who is told "no HDMI cable anywhere" when one sits in an unreachable lab has been actively misled.

THE HONEST GAP

State lives in memory only. A node that crashes forgets every reservation it granted. Persistence and recovery are Unit 3/4 material — list this on the FA-2 slide rather than waiting to be asked.

5 Traces and logical time

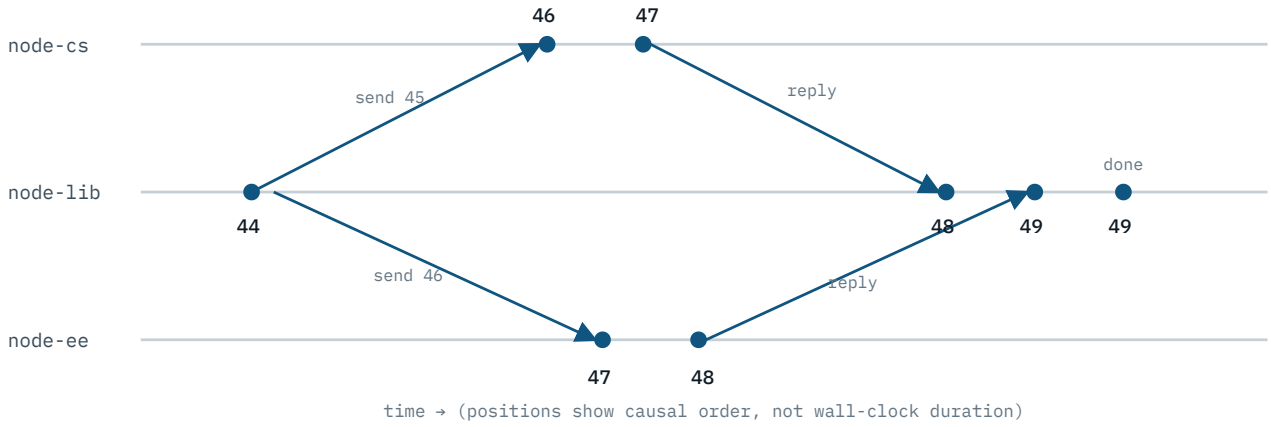
Real output from a running mesh, and the space–time diagram built from it.

One distributed search

```
captured from a live three-node mesh

[node-lib ] L=44   RPC OUT Search("arduino") -> node-cs, node-ee
[node-cs  ] L=46   RPC IN  Search("arduino") from node-lib -> 0 hit(s)
[node-ee  ] L=47   RPC IN  Search("arduino") from node-lib -> 1 hit(s)
[node-lib ] L=49   RPC OK  3/3 nodes responded in 10ms -> 1 match(es)
```

Read the clock values carefully, because they are the evidence. `node-lib` stamps 45 on the message to `node-cs` and 46 on the message to `node-ee` — two separate send events. `node-cs` receives 45 and moves to 46; `node-ee` receives 46 and moves to 47. Those two receives are **concurrent**: neither caused the other, and their numbers differ only because of the order the sends happened to be issued.



Every arrow is a happened-before edge. The two receives at 46 and 47 are concurrent — no path of arrows connects them. This is the diagram to put in the report for the Model of Distributed Computations topic.

One return, announced to the mesh

```
captured live — pub/sub, not request/reply

[node-ee  ] L=49   MQ OUT  campus/resource/available/iron  Soldering Iron 25W
[broker   ]        FWD     campus/resource/available/iron  (from node-ee-11188)
[node-ee  ] L=50   MQ IN   campus/resource/available/iron  Soldering Iron 25W (own echo)
[node-cs  ] L=50   MQ IN   campus/resource/available/iron  Soldering Iron 25W
[node-lib ] L=50   MQ IN   campus/resource/available/iron  Soldering Iron 25W
[node-cs  ] L=50   OK      1 student(s) waiting for "Soldering Iron 25W" - notifying
```

Three things to point out in this trace. `node-ee` published without addressing anyone. All three nodes received it, including `node-ee` itself — the own echo is kept deliberately, because another student on that same node might be the one waiting. And `node-cs` matched it against a parked request and notified a student who had asked nobody.

6 How FA-1 was built

Eight phases in dependency order, and what each one made possible.

#	PHASE	WHY IN THIS POSITION
1	One node alone — store, REST, UI	A node useful in isolation, so the next phase's leap is visible: identical UI, suddenly finding things it does not own.
2	**gRPC service + parallel fan-out**	The core of the project. Everything after this is a refinement of a working distributed search.
3	Docker — three containers, one image	Early on purpose. Every address in the codebase changes when you containerise; retrofitting after WebRTC lands is painful.
4	MQTT announcements + wantlist	Needs a working search first, because the wantlist exists precisely for the case where search finds nothing.
5	Heartbeat, peer state, staged failure	Needs peers to exist before liveness means anything.
6	WebRTC data channel	Needs both the reservation (which supplies the room) and the message bus (which relays signalling).
7	Media stream + QoS panel	Reuses phase 6's peer connection; only adds a track and a stats loop.
8	Lamport clocks, logs, report	Makes the invisible visible, once there is something to observe.

Verification

One command boots a broker and all three nodes, runs **71 checks** against the live mesh, kills a node to prove both partial results and heartbeat detection, restarts it to prove recovery, then tears everything down.

```
terminal

npm run verify              # standard ports
OFFSET=20 npm run verify    # shifted, to run beside a live demo mesh
```

The checks are grouped by the *concept* each one proves, so a failure names the broken property rather than just a line number. Re-run it after every change; it doubles as evidence in the report.

7 Unit 3 — Synchronization

DESIGNED FOR FA-2 · NOT YET BUILT

Clock synchronization, logical clocks, Lamport's algorithm, global state, vector clocks, election, mutual exclusion, beacon protocol.

Unit 3 is where Campus Mesh stops being a lookup system and starts being a *coordination* system. FA-1 left one honest hole: two students requesting the same item are serialised by whichever single node owns it. That hole is not a bug to patch — it is precisely the distributed mutual exclusion problem, and every other Unit 3 topic hangs off it.

THE SENTENCE THAT CONNECTS FA-1 TO FA-2

"FA-1 answers who has it. FA-2 answers who gets it — and that question cannot be settled without agreeing on order, on time, and on who decides."

CLOCK SYNCHRONIZATION

Physical clocks drift, and reservations have deadlines

Every machine has a quartz clock with a drift rate. Two machines synchronised today disagree tomorrow. Synchronisation algorithms differ in whether an authoritative time source exists:

- **Cristian's algorithm** — a client asks a time server, measures round-trip time, and sets its clock to the reply plus RTT/2 to compensate for the journey.
- **Berkeley algorithm** — no authoritative source. A coordinator polls everyone, computes an *average* (discarding outliers), and sends each node an *adjustment* rather than a time.
- **NTP** — a hierarchy of strata with statistical filtering, used in practice.

One rule matters more than the algorithm: a clock is never set *backwards*. Time running backwards breaks causality and file timestamps alike, so a fast clock is **slowed** until it converges.

> PLANNED FOR CAMPUS MESH
>
> A reservation is granted "for 2 hours". Whose two hours? `node-ee` grants at its own 14:03; `node-lib` shows the student an expiry computed against a clock that may be forty seconds out. Nobody notices until an item is marked overdue while the holder insists they are on time.
>
> Berkeley is the right fit, because no node here is authoritative — three departments, no head office. The elected coordinator polls, averages, and issues adjustments.
>
> **How to demo it:** deliberately skew one container's clock, show the three nodes disagreeing about an expiry time, run the algorithm, show them converge.

BUILT IN FA-1 LOGICAL CLOCKS · LAMPORT'S ALGORITHM

Ordering without a clock

Covered in depth in section 3 and already running: every gRPC and MQTT message carries a `lamport` value, and section 5 shows a real trace turned into a space–time diagram.

One addition FA-2 needs: Lamport numbers alone can tie. Two different nodes can legitimately hold the same value, as the real trace shows. A **total order** is obtained by breaking ties with the node identifier — the pair `(L, node_id)` is unique and consistent everywhere. Mutual exclusion depends on exactly this.

total ordering for the request queue

```
// Lamport value first; node identifier only to break a tie.
function earlier(a, b) {
  if (a.lamport !== b.lamport) return a.lamport < b.lamport;
  return a.node_id < b.node_id;
}
```

`VECTOR ALGORITHM (VECTOR CLOCKS)`

Detecting concurrency, not just ordering it

A Lamport clock guarantees *a → b implies L(a) < L(b)*, but not the converse. Seeing L(a) < L(b) tells you nothing about whether a actually caused b. For detecting a genuine conflict, that is useless.

A **vector clock** fixes it. Each node keeps a vector with one counter per node:

- On a local or send event, node *i* increments `V[i]`.
- On receive, it takes the element-wise maximum of its own vector and the sender's, then increments `V[i]`.

Then *a → b* exactly when `V(a) ≤ V(b)` element-wise and they differ somewhere. If neither vector dominates the other, the events are **concurrent** — and now you can prove it rather than guess.

> `PLANNED FOR CAMPUS MESH`
>
> Two students on two different nodes request the Arduino within the same second. With Lamport values you can order them, but you cannot tell whether one knew about the other. With vector clocks you can say definitively: *these two requests are concurrent* — which is the precise condition that requires arbitration rather than a simple first-come rule.
>
> Cost: one integer per node per message. The `lamport` field becomes a `vclock` map, and `src/clock.js` is the only file that changes shape.

GLOBAL STATE · CHANDY–LAMPORT SNAPSHOT

Photographing a system that will not hold still

"What does the whole campus hold right now?" has no obvious answer: by the time node C replies, node A has changed. Asking each node in turn gives an **inconsistent cut** — a picture that never actually existed, possibly showing an item in two places or nowhere.

The **Chandy–Lamport algorithm** records a consistent global snapshot without stopping the system:

1. An initiator records its own local state and sends a **marker** on every outgoing channel.
2. A node receiving its first marker records its own state immediately, then sends markers on all its outgoing channels.
3. Messages arriving on a channel between recording the state and receiving that channel's marker are recorded as **in-flight channel state**.

The result is a *consistent cut*: it may never have been the true instantaneous state, but it is a state the system could have been in, which is enough for detecting stable properties — deadlock, termination, lost items.

> PLANNED FOR CAMPUS MESH
>
> A campus-wide inventory report: every item, its holder, and every reservation still travelling between nodes — captured without pausing anybody's search. The in-flight channel state is the interesting part, and it is visible here: a `Reserve` that has left `node-lib` but not yet reached `node-ee` is a real item in a real limbo, and the snapshot must account for it or the totals will not add up.
>
> **How to demo it:** print the snapshot, show the per-node states plus the in-flight messages, and show the totals reconcile.

ELECTION ALGORITHMS

Choosing a coordinator, repeatedly

Several jobs need exactly one node doing them: the Berkeley time master, the snapshot initiator, a centralised lock manager. When that node dies, the survivors must agree on a replacement — without a central authority to appoint one.

ALGORITHM	MECHANISM	MESSAGES	CHARACTER
Bully	A node noticing the coordinator is gone sends ELECTION to all higher ids. Any higher id replies and takes over. Silence means it wins, and it announces COORDINATOR.	$O(n^2)$ worst case	Fast when the highest live id notices first; noisy otherwise.
Ring	An ELECTION message circulates the logical ring accumulating ids; the highest id in the completed circuit wins.	$O(n)$	Fewer messages, but one lap is needed before anyone knows.

> PLANNED FOR CAMPUS MESH
>
> The trigger already exists. FA-1's heartbeat marks a peer `DOWN` after three missed beats — that is the failure detector an election needs, and it is already running. If the dead peer is the current coordinator, start an election.
>
> The elected coordinator holds the time-master role and initiates snapshots. **Implement Bully**, because the project only has three nodes and the demo is sharper: `docker stop` the coordinator and watch the other two negotiate a successor in the log.
>
> `extends src/rpc/peers.js · startHeartbeat()`

MUTUAL EXCLUSION ALGORITHMS

The centrepiece: who gets the Arduino?

Only one student may hold an item. Granting a reservation is therefore a **critical section**, and the classic three approaches each answer "who decides?" differently.

APPROACH	HOW IT WORKS	MESSAGES PER ENTRY	WEAKNESS
Centralised	One coordinator holds a queue; request, grant, release.	3	Single point of failure; coordinator is a bottleneck.
Token ring	A token circulates; holding it grants entry.	1 to ∞	A lost token must be regenerated; latency even when idle.
Ricart–Agrawala (distributed)	Broadcast a timestamped request; enter only after every other node replies. A node defers its reply if it wants the section itself with an earlier timestamp.	2(n−1)	Every node must be reachable; one failure blocks everyone.

Ricart–Agrawala is where the Lamport total order earns its keep: ties are broken by `(L, node_id)`, which every node evaluates identically, so all nodes agree on who goes first without consulting each other.

> PLANNED FOR CAMPUS MESH
>
> Implement **both** the centralised scheme (using the elected coordinator) and Ricart–Agrawala, behind the one function FA-1 already isolated — `store.reserve()`. Then compare them on the same double-booking race:
>
> - message count per reservation, measured from the logs;
> - what happens when the coordinator dies mid-request — the centralised scheme stalls until an election completes, Ricart–Agrawala never had a coordinator to lose;
> - what happens when any one node dies — now the comparison reverses, and Ricart–Agrawala is the one that blocks.
>
> A comparison with numbers from your own system is worth far more than implementing one algorithm correctly.
>
> `replaces the body of store.reserve() · src/store.js`

THE DEMO THAT CLOSES THE FA-1 LOOP

In FA-1 you say "two simultaneous requests are serialised by one node, and that is the limitation". In FA-2 you fire both requests at *different* nodes simultaneously, show the vector clocks proving they are concurrent, show the algorithm ordering them, and show exactly one grant and one refusal — with no node privileged over another.

BEACON PROTOCOL

Periodic presence announcements

A beacon is a small message a node emits at a fixed interval to announce it exists, typically carrying a little state — identity, load, capability. Receivers infer membership from beacons arriving, and failure from beacons stopping. It underpins presence in wireless and mobile distributed systems, and generalises the heartbeat.

The design trade-off is the interval: frequent beaconing detects failure quickly but costs bandwidth and battery; infrequent beaconing is cheap but slow to notice a departure.

> PARTLY BUILT, EXTENDED IN FA-2
>
> FA-1's heartbeat is a point-to-point `Ping` — a liveness probe, not a beacon. FA-2 generalises it into a true beacon broadcast on `campus/node/beacon/<node_id>` carrying identity, inventory summary, current load and the coordinator the node believes is in charge.
>
> That last field is what makes it more than cosmetic: disagreement about the coordinator becomes visible in the beacons and triggers an election.

SELF STUDY FAIR MUTUAL EXCLUSION

Lodha and Kshemkalyani's fairness improvement

Ricart–Agrawala is correct but not especially *fair*: its ordering follows logical timestamps, and a node that is persistently unlucky with tie-breaks can be overtaken repeatedly. Lodha and Kshemkalyani's algorithm improves on it by having nodes carry additional information about pending requests, so entry order tracks the order requests were genuinely made, and it does so with fewer messages than Ricart–Agrawala in the common case.

> WHY IT MATTERS HERE SPECIFICALLY
>
> Fairness is not abstract in a resource-sharing system. If ties are broken by node identifier, every tie goes to `node-cs` before `node-ee` — so Computer Science students would systematically beat Electronics students to contested equipment. That is a real unfairness produced by an arbitrary implementation detail, and it is a good thing to notice out loud in a viva.

SELF STUDY DISTRIBUTED DEADLOCK DETECTION

Knapp's classification, and a deadlock you can actually stage

Deadlock needs four conditions together: mutual exclusion, hold-and-wait, no preemption, and circular wait. In a distributed system no node sees the whole **wait-for graph**, so detecting the cycle is itself a distributed problem. Knapp classifies the approaches into four families:

FAMILY	IDEA
Path-pushing	Nodes exchange fragments of the wait-for graph until someone sees a cycle.
Edge-chasing	A blocked process sends a probe along its waiting edges; if the probe returns to its originator, there is a cycle. (Chandy–Misra–Haas.)
Diffusing computation	A query spreads outward and replies fold back; the result reveals whether the initiator is deadlocked.
Global state detection	Build a consistent snapshot — Chandy–Lamport — and look for the cycle in it.

> THE SCENARIO THIS PROJECT GIVES YOU FOR FREE
>
> A student needs *both* a projector and an HDMI cable, and so does another student.
>
> - Student A holds `node-cs:projector:0001` and waits for `node-ee:cable:0001`.
> - Student B holds `node-ee:cable:0001` and waits for `node-cs:projector:0001`.
>
> A circular wait spanning two nodes, where neither node can see the cycle alone. This is not a contrived example — it is what actually happens when people book equipment for a presentation.
>
> **Implement edge-chasing** (Chandy–Misra–Haas): the blocked node sends a probe carrying *(initiator, sender, receiver)* along the waiting edge; if a probe comes back to its initiator, a cycle exists. Resolve by aborting the *younger* request — using the Lamport timestamp to decide which is younger, which ties the whole unit together.
>
> Note that the last family in the table reuses the snapshot you already built, so you can detect the same deadlock two different ways and compare.

8 Unit 4 — Emerging distributed paradigms

DESIGNED FOR FA-2 · NOT YET BUILT

Object-based systems, web-based systems, distributed file systems, serverless architectures, and the case studies.

Unit 3 deepens the system. Unit 4 widens it — placing Campus Mesh in the landscape of how distributed systems are actually built today, and adding the pieces that make it look like production software rather than a lab exercise.

DISTRIBUTED OBJECT-BASED SYSTEMS

From remote procedures to remote objects

An RPC call is stateless from the interface's point of view: you call a procedure, you get a result. A **distributed object** has identity and state, and you invoke methods *on a particular object* — so the system must carry object references across the network, bind to them, and decide where the state lives.

The vocabulary to have ready:

- **Remote object reference** — a network-wide identifier for one object.
- **Proxy and skeleton** — the client-side stand-in and the server-side dispatcher.
- **Static vs dynamic invocation** — interfaces known at compile time, versus composed at run time.
- **Persistent vs transient objects** — whether the object outlives the server process.
- Classic realisations: **Java RMI**, **CORBA**, and today's gRPC-plus-service-mesh designs.

> WHERE CAMPUS MESH ALREADY IS, AND WHERE IT GOES
>
> Today the design is procedural: `Reserve(rid)` passes an identifier to a service. The object-based reframing is that **every resource is a distributed object** — it already has a network-wide reference (`node-ee:cable:0001`), state (status, holder, version), and methods (reserve, release, inspect). The owning node is its *home*.
>
> FA-2 makes that explicit: a `ResourceObject` service where a client binds to one resource reference and invokes methods on it, with the proxy hiding which node hosts it. The identifier scheme built in FA-1 is what makes this a small change rather than a redesign.

DISTRIBUTED WEB-BASED SYSTEMS

The web as a distributed system

The web is the largest distributed system in existence, and its architecture is worth naming precisely: stateless HTTP request–response, resources identified by URI, representations negotiated between client and server, and scale achieved through **caching and replication at every layer** — browser cache, proxy, reverse proxy, CDN edge.

REST's constraints — statelessness, uniform interface, cacheability — exist to make that layered caching possible. Statelessness is what lets any replica serve any request.

> PLANNED FOR CAMPUS MESH
>
> The browser-facing half of each node is already a web-based system: stateless REST, plus Server-Sent Events for push. FA-2 adds the parts that make it a *scalable* one — a short-TTL cache of search results with explicit invalidation when a `reserved` or `available` announcement arrives, and a reverse proxy in front of the node replicas.
>
> The interesting trade-off to discuss: caching a search result makes the system faster and staler at the same time. For a resource that may be taken at any moment, how long is too long? That is the same freshness-versus-load question a CDN answers, at a scale you can reason about.

DISTRIBUTED FILE SYSTEMS

Files that outlive the machine holding them

A distributed file system presents files stored across many machines as one namespace. The design questions are always the same four: **naming**, **caching**, **replication**, and **consistency semantics**.

SYSTEM	SHAPE	CONSISTENCY
NFS	Client–server, mounted remote directories	Weak; client caching with periodic revalidation
AFS	Whole-file caching with callbacks	Session semantics — changes visible on close
HDFS	NameNode holds metadata, DataNodes hold 128 MB blocks, replication factor 3	Write-once, read-many — deliberately restricted

HDFS is worth understanding specifically because your syllabus pairs it with MapReduce: the replication factor of 3 is not only durability, it is what lets computation be scheduled *on the node that already holds the data*.

> PLANNED FOR CAMPUS MESH
>
> Every resource gets a photo and, where relevant, a manual — so a student can see the exact adapter before walking over, and so a damaged item can be evidenced on return. These files must survive the owning node going down, which is precisely what FA-1 cannot do today.
>
> Store them with a replication factor of 2 across the three nodes: the owner plus one neighbour. A `GetFile` RPC falls back to the replica when the owner is `DOWN` — demonstrable by stopping a container and watching the photo still load.
>
> Naming reuses the resource identifier, so the file for `node-ee:cable:0001` needs no separate directory service.

SERVERLESS ARCHITECTURES

Functions without a server to manage

Serverless does not mean no servers; it means the developer does not provision them. A **Function as a Service** platform runs a stateless function in response to an event, scales instances with demand — including down to zero — and bills per invocation.

The properties that follow, and the trade-offs that come with them:

- **Event-driven** — functions are triggered, not called; this pairs naturally with publish/subscribe.
- **Stateless** — any state must live outside the function, which is a real constraint, not a detail.
- **Cold starts** — scaling from zero costs latency on the first request.
- **Vendor coupling** — the trigger and runtime model are rarely portable.

> PLANNED FOR CAMPUS MESH
>
> Two jobs fit the model exactly, and both are already event-shaped because FA-1 is built on pub/sub:
>
> - **Thumbnailing** — a photo upload triggers a function that produces a small version. Bursty, stateless, embarrassingly parallel.
> - **Digest notifications** — an `available` announcement triggers a function that emails everyone whose wantlist matched. Idle most of the day, bursty at lab-closing time.
>
> Run them on Knative or OpenFaaS on the same Kubernetes cluster, so no cloud account is needed. The honest observation to make: the *node* itself is a poor serverless candidate, because it holds state and long-lived gRPC connections. Knowing what should *not* be serverless is the more interesting half of the topic.

CASE STUDIES

Megaport, Cloudflare, AWS, Hadoop, Kubernetes

Each case study illustrates a different distributed-systems idea. Tie each one to something in your own system rather than reciting marketing copy.

CASE	THE IDEA IT ILLUSTRATES	LINK TO CAMPUS MESH
Kubernetes	Declarative orchestration: you describe desired state, a control loop reconciles reality with it. Pods, Services, Deployments, self-healing, service discovery.	Direct upgrade from Compose. Three Deployments and three Services; kill a pod and watch it be recreated — a stronger fault-tolerance demo than `docker stop`.
Apache Hadoop	Move computation to the data. MapReduce over HDFS blocks, with the scheduler favouring nodes already holding the block.	A MapReduce job over reservation logs: most-demanded category per department, peak hours, average holding time. The map emits *(category, 1)* — the word-count pattern applied to real data you generated.
Cloudflare	Edge computing and anycast: serve and compute close to the user; one address, many physical locations.	The conceptual argument for putting a node in each building rather than one in the data centre — latency and partition tolerance, the same reasoning at campus scale.
AWS	Regions and availability zones; managed services as building blocks; the shared responsibility model.	Where this would be deployed if the campus did not own hardware, and what you would hand over: queueing to SQS, the file store to S3, functions to Lambda.
Megaport	Network as a Service — software-defined private interconnection between clouds and data centres, provisioned on demand instead of as physical circuits.	The layer beneath everything else: your overlay assumes connectivity between departments. Megaport is that assumption made into a product, and it is the right place to mention that an overlay is only as good as the network under it.

SELF STUDY BLOCKCHAIN & DISTRIBUTED LEDGERS

Consensus without trust — and whether you need it

A distributed ledger is an append-only, replicated, tamper-evident log. Each block carries the hash of its predecessor, so altering any record invalidates every record after it. Agreement on what to append comes from a consensus mechanism — Proof of Work, Proof of Stake, or for permissioned systems a Byzantine-fault-tolerant protocol like PBFT.

The property it buys is specific and expensive: agreement among parties who **do not trust each other** and share no authority.

> THE HONEST ANALYSIS — WORTH MORE THAN AN IMPLEMENTATION
>
> The plausible use is a tamper-evident reservation history: if Electronics claims a Library student damaged a multimeter, nobody can quietly rewrite who held it.
>
> But the counter-argument is stronger, and making it yourself is the better answer. The three departments are part of one institution with a shared authority. They are not mutually distrusting strangers. A **hash-chained append-only log**, signed by the owning node, gives the same tamper-evidence at a fraction of the cost — no consensus round, no mining, no latency.
>
> Knowing when *not* to reach for blockchain is the point of the topic.

SELF STUDY · DISTRIBUTED DATABASE TRADE-OFFS

CAP, PACELC, and the choice you are actually making

CAP: during a network *partition*, a system must choose between consistency and availability. It cannot have both, and the theorem says nothing about the normal case. **PACELC** completes the picture: if Partitioned, choose Availability or Consistency; **E**lse, choose **L**atency or **C**onsistency. Most real systems trade consistency for latency every day, long before any partition.

Related choices: **sharding** (partitioning data for scale) versus **replication** (copying it for availability); and the consistency model offered — strong, sequential, causal, or eventual.

> CAMPUS MESH'S POSITION, STATED PLAINLY
>
> For reservations, Campus Mesh must be **CP**. If `node-ee` is unreachable, the correct behaviour is to refuse the reservation — which is exactly what FA-1 does with its 503 — because granting it anyway risks two students arriving for one Arduino. Availability is the right thing to sacrifice when the alternative is being wrong.
>
> For *search*, the opposite choice is correct: return partial results from reachable nodes, labelled "2 of 3 responded". AP for reads, CP for writes, in the same system, decided per operation.
>
> Being able to say that sentence about your own system is the whole topic. The syllabus suggests MongoDB; a replica set with majority write concern is the CP configuration, and it is worth naming why you would choose it over the default.

9 The FA-2 build plan

Same project, same demo, two new units. Eleven steps in dependency order.

FA-2 is not a new system. Every step below extends code that already exists, and the three hooks left in FA-1 — a clock on every message, unused `version` / `lockedBy` / `lockExpiry` fields, and all reservation logic behind one function — are what make the first six steps cheap.

Part A — Synchronization (Unit 3)

#	STEP	WHAT IT PROVES
A1	Vector clocks replace the Lamport scalar on the wire	Two requests are provably *concurrent*, not merely ordered
A2	Total order `(L, node_id)` for the request queue	All nodes agree on precedence without consulting each other
A3	**Ricart–Agrawala mutual exclusion** around `store.reserve()`	The double-booking race, solved with no coordinator
A4	Centralised mutual exclusion as the comparison arm	Message counts and failure behaviour, measured both ways
A5	Bully election, triggered by the existing heartbeat	The coordinator role survives the coordinator dying
A6	Berkeley clock synchronization under the elected master	Three skewed clocks converge; expiry times agree
A7	Chandy–Lamport snapshot → campus inventory report	A consistent cut, including in-flight reservations
A8	Edge-chasing deadlock detection over the wait-for graph	A cross-node circular wait, detected and broken
A9	Beacon protocol generalising the heartbeat	Presence, load and coordinator belief, broadcast

Part B — Emerging paradigms (Unit 4)

#	STEP	WHAT IT PROVES
B1	Kubernetes deployment replacing Compose	Declarative orchestration; kill a pod, watch it return
B2	Replicated file store for item photos (factor 2)	A file survives its owning node going down
B3	Serverless functions for thumbnails and digests	Event-driven, stateless, scale-to-zero
B4	Hadoop MapReduce over the reservation logs	Move computation to the data; real demand analytics
B5	Resources exposed as distributed objects	Remote object references rather than remote procedures
B6	Search-result caching with announcement-driven invalidation	Web-scale caching, and the staleness it costs

IF TIME RUNS SHORT

A1, A3, A5 and B1 are the spine. Vector clocks prove the conflict is real, Ricart–Agrawala resolves it, Bully keeps the system running when a node dies, and Kubernetes makes the whole thing demonstrable. Everything else is depth on top of a complete story.

The FA-2 demo, in four minutes

MIN	ACTION	CONCEPT NAMED ALOUD
0:00	Recap FA-1 in one search, then state the limitation it left	Continuity
0:45	Fire two reservations for one item at two different nodes	The race, reproduced on demand
1:15	Show the vector clocks: neither dominates — genuinely concurrent	Vector clocks, causality
1:45	Watch Ricart–Agrawala order them: one grant, one refusal	Distributed mutual exclusion
2:30	Kill the coordinator; watch the election in the log	Bully algorithm, failure detection
3:00	Stage the projector/cable circular wait; probe finds the cycle	Edge-chasing deadlock detection
3:30	Run the snapshot; totals reconcile including in-flight requests	Consistent global state
4:00	Hadoop job output: most-demanded equipment per department	MapReduce, data locality

10 Demo script

Seven minutes. Rehearse it three times on the machine you will present from, and keep a screen recording as backup.

MIN	ACTION	CONCEPT YOU NAME ALOUD
0:00	`docker compose up` — three UIs and the log side by side	Architecture, virtualization
0:45	Search `hdmi` from the Library — found in Electronics	RPC, location transparency
1:30	Point at the log: fan-out, Lamport values, latency	Middleware, logical clocks
2:15	`docker stop node-ee`, search again — "2 of 3 responded"	Fault tolerance, honest degradation
3:00	`docker start node-ee` — the dot turns green unprompted	Heartbeat, self-healing
3:30	Notify me on a borrowed item, then Return it elsewhere	Message-oriented, pub/sub decoupling
4:30	Request an item — handover channel opens between two windows	P2P, signalling vs data
5:15	Show the item on camera; throttle the network; QoS degrades	Stream-oriented, multimedia QoS
6:15	Show `resource.proto` — "a fourth node joins by implementing this"	Openness, scalability
6:45	One slide: what FA-2 adds	Continuity

THE STRONGEST THIRTY SECONDS YOU HAVE

Once the handover channel is open between two windows, **stop every node container and the broker**, then keep chatting. The nodes are gone and the conversation continues. Have this rehearsed — it is the single most convincing moment in the whole demo.

11 Viva questions

Both team members are questioned separately about the whole system. These are the ones that actually get asked.

<details>
<summary>Is this actually distributed, or three programs on one laptop?</summary>

No shared memory, no shared database, no coordinator. Each node owns its data and can learn about another node's inventory only by sending it a message. Containers give each its own network identity and filesystem. The topology is unchanged if they move to three machines — only the `PEERS` addresses change, not one line of logic.
</details>

<details>
<summary>What happens if two students request the same item at the same moment?</summary>

The owning node serialises the two requests and the second gets a refusal, because a single node is a single point of serialisation. That is exactly the limitation FA-2 addresses with distributed locking and replication. **Do not claim this is solved** — naming the limitation shows you understand the problem; claiming a solution invites a question you cannot win.
</details>

<details>
<summary>Why not just one central server?</summary>

Three reasons, and they are the same reasons real systems decentralise. Departments own their inventory and will not surrender it to a central authority. A central server is a single point of failure. And each department must keep working during a network partition.
</details>

<details>
<summary>Where is the middleware?</summary>

`src/rpc/peers.js` and `src/mq/bus.js`. The application calls `distributedSearch("hdmi")` and never touches sockets, serialisation, retries, name resolution or timeouts. Everything above those files speaks in resources and queries; everything below speaks in bytes.
</details>

▼ Why four different communication mechanisms? Is that not over-engineering?

Each answers a different question. **gRPC** because a student is waiting for an answer and a reply is required. **MQTT** because nobody is waiting, and a student returning an item should not block on unknown listeners. **WebRTC data** because it is two students, not two nodes, and routing through a server adds latency and a failure point for no benefit. **WebRTC media** because continuous media has deadlines, and a late frame is discarded rather than retransmitted.

▼ What are your RPC failure semantics?

`Search` is idempotent, so retrying is safe. `Reserve` is not — a blind retry could double-book — so it is never retried, giving at-most-once semantics. A failed reserve surfaces as a 503 and the student decides. Guessing on their behalf is what would cause the bug.

▼ Is your messaging persistent?

No. QoS 0, no retained messages, so announcements are transient — a node that is offline when an item is returned never hears about it. Persistence would need QoS 1 plus a durable session, which is a clean FA-2 addition. This is a question worth pre-empting, because "we used MQTT" invites it.

▼ Prove the WebRTC channel is really peer-to-peer.

Open the handover channel between two windows, then stop every node container and the broker. `fetch('/api/node')` fails in both browsers, and messages still cross with the candidate-pair byte counters still climbing. `Handover.debug()` in the console prints connection state, channel state and those counters.

▼ Why Lamport clocks rather than timestamps?

Wall clocks on three machines disagree, so a timestamp cannot answer "which event happened first". A Lamport clock gives a partial order respecting causality: if *a* caused *b* then $L(a) < L(b)$. The converse does not hold — equal or ordered numbers do not prove causality — which is exactly why vector clocks come next.

▼ Your FA-1 serialises reservations on one node. How does FA-2 fix that?

By treating the grant as a critical section and applying distributed mutual exclusion. Ricart–Agrawala: the node broadcasts a timestamped request and enters only once every other node has replied, deferring its own reply if it wants the section with an earlier timestamp. Ties break on `(Lamport, node_id)`, which every node evaluates identically, so no coordinator is needed and no node is privileged.

▼ Why do you need vector clocks if you already have Lamport clocks?

Lamport guarantees that if *a* caused *b* then L(a) < L(b), but not the reverse — so a smaller number does not prove causality. To know whether two reservation requests are genuinely *concurrent*, you need vector clocks: if neither vector dominates the other element-wise, the events are concurrent, provably. That is the exact condition that requires arbitration rather than first-come-first-served.

▼ Which election algorithm, and why?

Bully. With three nodes the $O(n^2)$ message cost is irrelevant, and the demo is clearer: stop the coordinator's container and the survivors visibly negotiate a successor. The failure detector it needs already exists — FA-1's heartbeat marks a peer DOWN after three missed beats, and that is the trigger. Ring would use fewer messages but needs a full lap before anyone knows the outcome.

▼ Why can't you just ask every node what it holds to get a global state?

Because by the time the third node replies, the first has changed — you get an inconsistent cut, a picture that never existed, where an item can appear twice or vanish. Chandy–Lamport solves it with markers: record local state on the first marker, then propagate markers, recording anything that arrives on a channel before that channel's marker as in-flight state. In this system the in-flight part is real — a `Reserve` that has left one node but not arrived at another.

▼ Show me a deadlock that could actually occur here.

Two students each need a projector and an HDMI cable for a presentation. A holds the projector on `node-cs` and waits for the cable on `node-ee`; B holds that cable and waits for that projector. A circular wait spanning two nodes, where neither node can see the cycle on its own. Edge-chasing detects it: the blocked node sends a probe along the waiting edge, and a probe returning to its initiator proves a cycle. Resolve by aborting the younger request, using the Lamport timestamp to decide which is younger.

▼ Is Campus Mesh CP or AP?

Both, decided per operation. Reservations are **CP**: if the owning node is unreachable we refuse rather than risk two students arriving for one Arduino — that is the 503 in FA-1. Search is **AP**: return what the reachable nodes have, labelled "2 of 3 responded". Sacrificing availability is correct when the alternative is being wrong, and sacrificing consistency is correct when a partial answer is still useful.

▼ Would blockchain be useful for the reservation history?

Probably not, and that is the interesting answer. A ledger buys agreement among parties who do not trust each other and share no authority. Three departments of one institution do share an authority. A hash-chained append-only log signed by the owning node gives the same tamper-evidence with no consensus round, no mining and no added latency. Knowing when not to reach for it is the point of the topic.

▼ What in your system should *not* be serverless?

The node itself. It holds state and long-lived gRPC connections to its peers, and a scale-to-zero stateless function can do neither. Thumbnailing photos and sending digest notifications fit perfectly — event-triggered, stateless, bursty — and they are already event-shaped because FA-1 is built on publish/subscribe.

▼ How would a fourth department join?

Implement `proto/resource.proto`, point `PEERS` at the existing nodes, and add itself to theirs. No existing node changes its code. That is what openness means concretely, and it is why the IDL belongs in the report.

12 Limitations

State these before you are asked. A named limitation reads as understanding; a discovered one reads as an oversight.

LIMITATION	CONSEQUENCE	FA-2 DIRECTION
Single point of serialisation per item	Concurrent requests are ordered by whichever node owns the item	Distributed mutual exclusion — Ricart–Agrawala (A3)
Lamport, not vector, clocks	Concurrency can be ordered but not *detected*	Vector clocks (A1)
No elected coordinator	No node can hold a role that must survive a failure	Bully election (A5)
Physical clocks never synchronised	Nodes disagree about when a reservation expires	Berkeley algorithm (A6)
No global state capture	A campus-wide inventory count can never be consistent	Chandy–Lamport snapshot (A7)
No deadlock handling	A cross-node circular wait would simply hang	Edge-chasing detection (A8)
In-memory state only	A crashed node forgets every reservation it granted	Replicated file store and persistence (B2)
Transient messaging (QoS 0)	Offline nodes miss announcements entirely	QoS 1 with durable sessions
No authentication	A node could spoof another's identifier	Out of syllabus scope; worth naming anyway
STUN but no TURN	A restrictive network would block the peer link	A TURN relay

Hooks already in the code

Three things were built in early so that FA-2 costs little:

- Every gRPC and MQTT message carries a `lamport` value — upgrade to vector clocks and concurrency becomes *detectable*, not merely orderable.
- Every resource carries unused `version`, `lockedBy` and `lockExpiry` fields — optimistic concurrency control and lease-based locking with no schema change.
- All reservation logic sits behind `store.reserve()` — wrapping it in a mutual-exclusion protocol touches exactly one function.

THE FA-2 SENTENCE

"The double-booking race is the entry point. It turns granting a reservation into a critical section, which is distributed mutual exclusion — and that drags in logical time, total ordering, leader election and deadlock detection behind it. Same project, same demo, two more units of theory."
